## A. Eliminate punctuation using Python's string module

```
import string

text = "Hello! How are you doing today? I'm excited about NLP :) #AI"

clean_text = text.translate(
    str.maketrans("", "", string.punctuation)
)

print("Original:", text)
print("Without Punctuation:", clean_text)
```

```
Original: Hello! How are you doing today? I'm excited about NLP :) #AI
Without Punctuation: Hello How are you doing today Im excited about NLP  AI
```

## B. Eliminate punctuation using NLTK tokenization

```
import string
import nltk
from nltk.tokenize import word_tokenize

nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)

text = "Hello! How are you doing today? I'm learning NLP, it's amazing :)"

tokens = word_tokenize(text)

# Remove standalone punctuation tokens
clean_tokens = [
    token for token in tokens
    if not all(char in string.punctuation for char in token)
]

clean_text = " ".join(clean_tokens)

print("Original Text:", text)
print("Text without punctuation:", clean_text)
```

```
Original Text: Hello! How are you doing today? I'm learning NLP, it's amazing
Text without punctuation: Hello How are you doing today I 'm learning NLP it
```

## C. Convert text to lowercase

```
text = (
    "This is a Demo Text for NLP using NLTK. "
    "Full form of NLTK is Natural Language Toolkit"
)

lower_text = text.lower()

print(lower_text)
```

```
this is a demo text for nlp using nltk. full form of nltk is natural language
```

## D. Convert text to uppercase

```
text = (
    "This is a Demo Text for NLP using NLTK. "
    "Full form of NLTK is Natural Language Toolkit"
)

upper_text = text.upper()

print(upper_text)
```

```
THIS IS A DEMO TEXT FOR NLP USING NLTK. FULL FORM OF NLTK IS NATURAL LANGUAGE
```

## E. Expand abbreviations

```
import re

abbrev_dict = {
    "asap": "as soon as possible",
    "fyi": "for your information",
    "idk": "i don't know"
}

text = "FYI, I need it ASAP. IDK what happened."

pattern = re.compile(
    r"\b(" + "|".join(map(re.escape, abbrev_dict.keys())) + r")\b",
    flags=re.IGNORECASE
)

expanded_text = pattern.sub(
    lambda match: abbrev_dict[match.group(0).lower()],
    text
)
```

```
print("Original:", text)
print("Expanded:", expanded_text)
```

```
Original: FYI, I need it ASAP. IDK what happened.
Expanded: for your information, I need it as soon as possible. i don't know w
```

## F. Convert numbers into words

```
!pip -q install inflect
import re
import inflect

p = inflect.engine()

text = "I have 2 apples and 30 bananas."

converted_text = re.sub(
    r"\b\d+\b",
    lambda match: p.number_to_words(int(match.group(0))),
    text
)

print("Original:", text)
print("Converted:", converted_text)
```

```
Original: I have 2 apples and 30 bananas.
Converted: I have two apples and thirty bananas.
```

## G. Remove stop words

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)

text = (
    "This is a Demo Text for NLP using NLTK. "
    "Full form of NLTK is Natural Language Toolkit"
)

stop_words = set(stopwords.words("english"))
tokens = word_tokenize(text)

remaining_tokens = [
    word for word in tokens
```

```
    if word.lower() not in stop_words
]

print("Original Tokens:", tokens)
print("After Stop Word Removal:", remaining_tokens)
```

```
Original Tokens: ['This', 'is', 'a', 'Demo', 'Text', 'for', 'NLP', 'using', '
After Stop Word Removal: ['Demo', 'Text', 'NLP', 'using', 'NLTK', '.', 'Full'
```

## H. Stemming using Porter Stemmer

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer

nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)

stemmer = PorterStemmer()

text = (
    "It originated from the idea that there are readers who "
    "prefer learning new skills from the comforts of their drawing rooms"
)

tokens = word_tokenize(text)

for word in tokens:
    print(f"Actual: {word} | Stem: {stemmer.stem(word)}")
```

```
Actual: It | Stem: it
Actual: originated | Stem: origin
Actual: from | Stem: from
Actual: the | Stem: the
Actual: idea | Stem: idea
Actual: that | Stem: that
Actual: there | Stem: there
Actual: are | Stem: are
Actual: readers | Stem: reader
Actual: who | Stem: who
Actual: prefer | Stem: prefer
Actual: learning | Stem: learn
Actual: new | Stem: new
Actual: skills | Stem: skill
Actual: from | Stem: from
Actual: the | Stem: the
Actual: comforts | Stem: comfort
Actual: of | Stem: of
Actual: their | Stem: their
Actual: drawing | Stem: draw
Actual: rooms | Stem: room
```

## I. Lemmatization using NLTK

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('omw-1.4', quiet=True)

lemmatizer = WordNetLemmatizer()

words = ["cars", "running", "studies", "better"]

for word in words:
    print(f"Original: {word} | Lemma: {lemmatizer.lemmatize(word)}")
```

```
Original: cars | Lemma: car
Original: running | Lemma: running
Original: studies | Lemma: study
Original: better | Lemma: better
```

```
for word in ["running", "studies"]:
    print(word, "->", lemmatizer.lemmatize(word, pos="v"))
```

```
running -> run
studies -> study
```

## J. Canonicalization

```
import unicodedata

text = "café"

canonical_text = unicodedata.normalize("NFC", text)

print("Original:", text)
print("Canonicalized:", canonical_text)
```

```
Original: café
Canonicalized: café
```